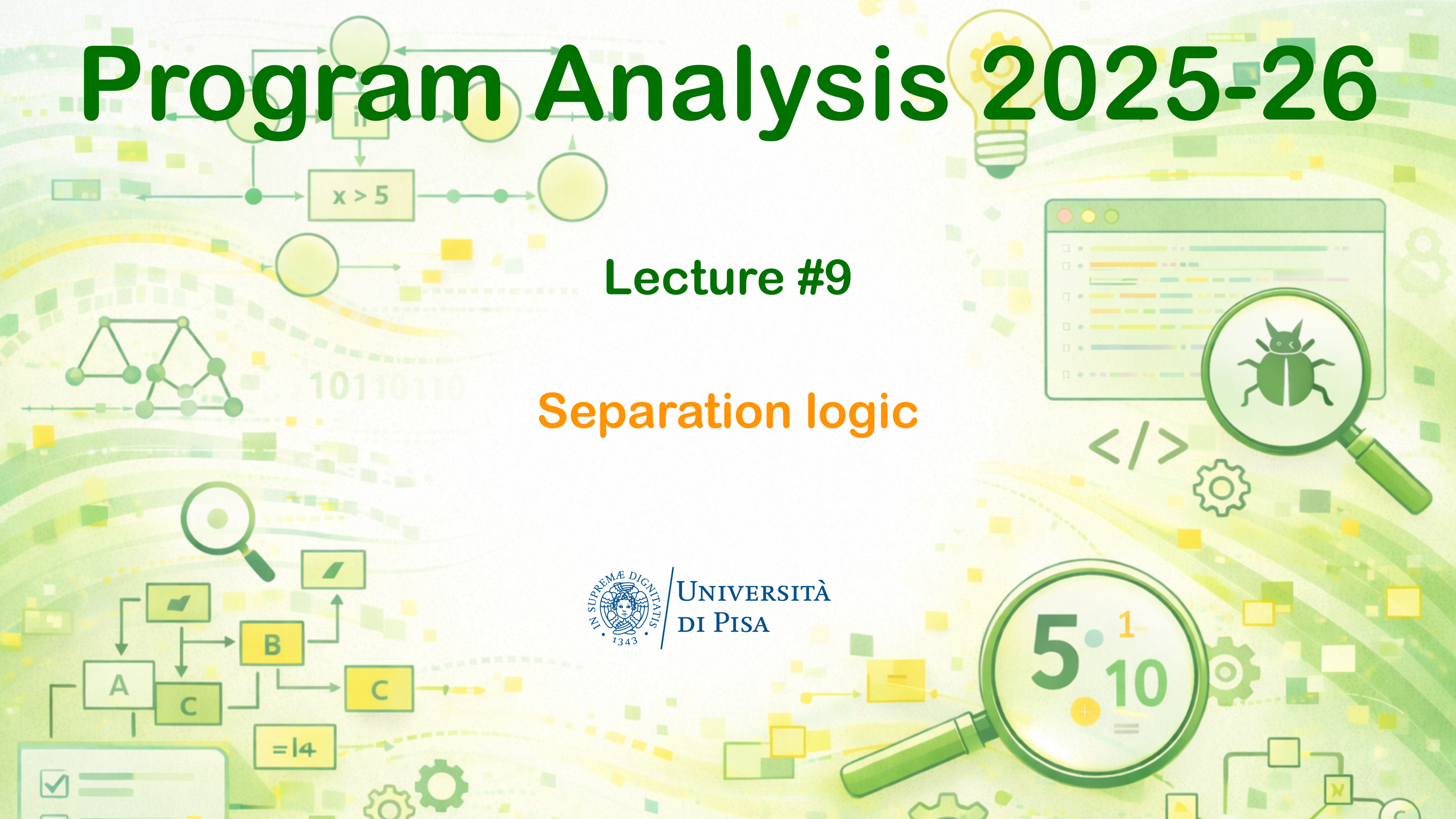
Program Analysis 2025-26

Lecture #9

Separation logic



Università di Pisa

The taxonomy

	Forward	Backward
Over	HL $\{P\}\ c\ \{Q\}$ $[\![c]\!]P \subseteq Q$ $\forall \sigma \in P\,.\,\forall \delta \in [\![c]\!]\sigma\,.\,\delta \in Q$	NC $(P)\ c\ (Q)$ $P \supseteq [\![\overleftarrow{c}]\!]Q$ $\forall \delta \in Q\,.\,\forall \sigma \in [\![\overleftarrow{c}]\!]\delta\,.\,\sigma \in P$
Under	IL $[P]\ c\ [Q]$ $[\![c]\!]P \supseteq Q$ $\forall \delta \in Q\,.\,\exists \sigma \in P\,.\,\delta \in [\![c]\!]\sigma$	SIL $\langle P \rangle\ c\ \langle Q \rangle$ $P \subseteq [\![\overleftarrow{c}]\!]Q$ $\forall \sigma \in P\,.\,\exists \delta \in Q\,.\,\delta \in [\![c]\!]\sigma$

Code summaries

// function r

```
x := nondet();
if (x=1) {
  if (y≤100) { MC }
}
```

// function MC is the McCarthy 91 function

```
while (x>0) {
   if (y>100) {
       y := y-10; x := x-1 }
   else {
       y := y+11; x := x+1 } }
```

$$f(y) = \begin{cases} y - 10 & \text{se } y > 100 \\ 91 & \text{se } y \leq 100 \end{cases}$$

	SIL	IL	HL	NC
$[\text{true}]\ \mathsf{r}\ [y = 91 \wedge x \neq 1]$				
$\langle\!\langle y \leq 100 \rangle\!\rangle\ \mathsf{r}\ \langle\!\langle y = 91 \wedge x \neq 1 \rangle\!\rangle$				
$\langle\!\langle y \leq 100 \rangle\!\rangle\ \mathsf{r}\ \langle\!\langle y = 91 \rangle\!\rangle$				
$\langle\!\langle y < 91 \rangle\!\rangle\ \mathsf{r}\ \langle\!\langle y = 91 \rangle\!\rangle$				

Code summaries

// function r

```
x := nondet();
if (x=1) {
   if (y≤100) { x := 0;
                y := 91 }
}
```

// function MC is the McCarthy 91 function

```
while (x>0) {
   if (y>100) {
        y := y-10; x := x-1 }
   else {
        y := y+11; x := x+1 } }
```

$$f(y) = \begin{cases} y - 10 & \text{se } y > 100 \\ 91 & \text{se } y \leq 100 \end{cases}$$

	SIL	IL	HL	NC
$[\text{true}]\ \mathsf{r}\ [y = 91 \wedge x \neq 1]$				
$\langle\!\langle y \leq 100 \rangle\!\rangle\ \mathsf{r}\ \langle\!\langle y = 91 \wedge x \neq 1 \rangle\!\rangle$				
$\langle\!\langle y \leq 100 \rangle\!\rangle\ \mathsf{r}\ \langle\!\langle y = 91 \rangle\!\rangle$				
$\langle\!\langle y < 91 \rangle\!\rangle\ \mathsf{r}\ \langle\!\langle y = 91 \rangle\!\rangle$				

Code summaries

// function r

```
x := nondet();
if (x=1) {
   if (y≤100) { x := 0;
                y := 91 }
}
```

// function MC is the McCarthy 91 function

```
while (x>0) {
   if (y>100) {
        y := y-10; x := x-1 }
   else {
        y := y+11; x := x+1 } }
```

$$f(y) = \begin{cases} y - 10 & \text{se } y > 100 \\ 91 & \text{se } y \leq 100 \end{cases}$$

	SIL	IL	HL	NC
$[\text{true}]\ \mathsf{r}\ [y = 91 \wedge x \neq 1]$	✗	✓	✗	✓
$\langle\!\langle y \leq 100 \rangle\!\rangle\ \mathsf{r}\ \langle\!\langle y = 91 \wedge x \neq 1 \rangle\!\rangle$	✓	✓	✗	✓
$\langle\!\langle y \leq 100 \rangle\!\rangle\ \mathsf{r}\ \langle\!\langle y = 91 \rangle\!\rangle$	✓	✗	✗	✓
$\langle\!\langle y < 91 \rangle\!\rangle\ \mathsf{r}\ \langle\!\langle y = 91 \rangle\!\rangle$	✓	✗	✗	✗

MEMORY SAFETY NOTICE

This presentation contains pointers.

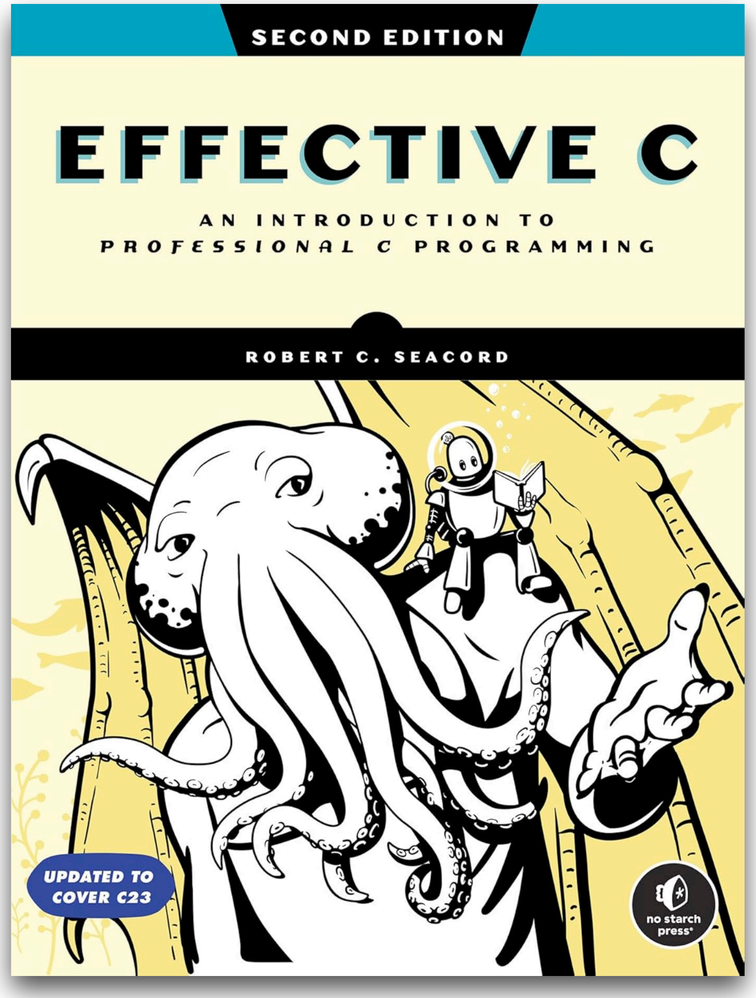
Attend at your own risk.

Pointer analysis

Back to HL, but with pointers

	Forward
Over	HL $\{P\}\ c\ \{Q\}$ $[\![c]\!]P \subseteq Q$ $\forall \sigma \in P\,.\,\forall \delta \in [\![c]\!]\sigma\,.\,\delta \in Q$

Why is pointer analysis important?



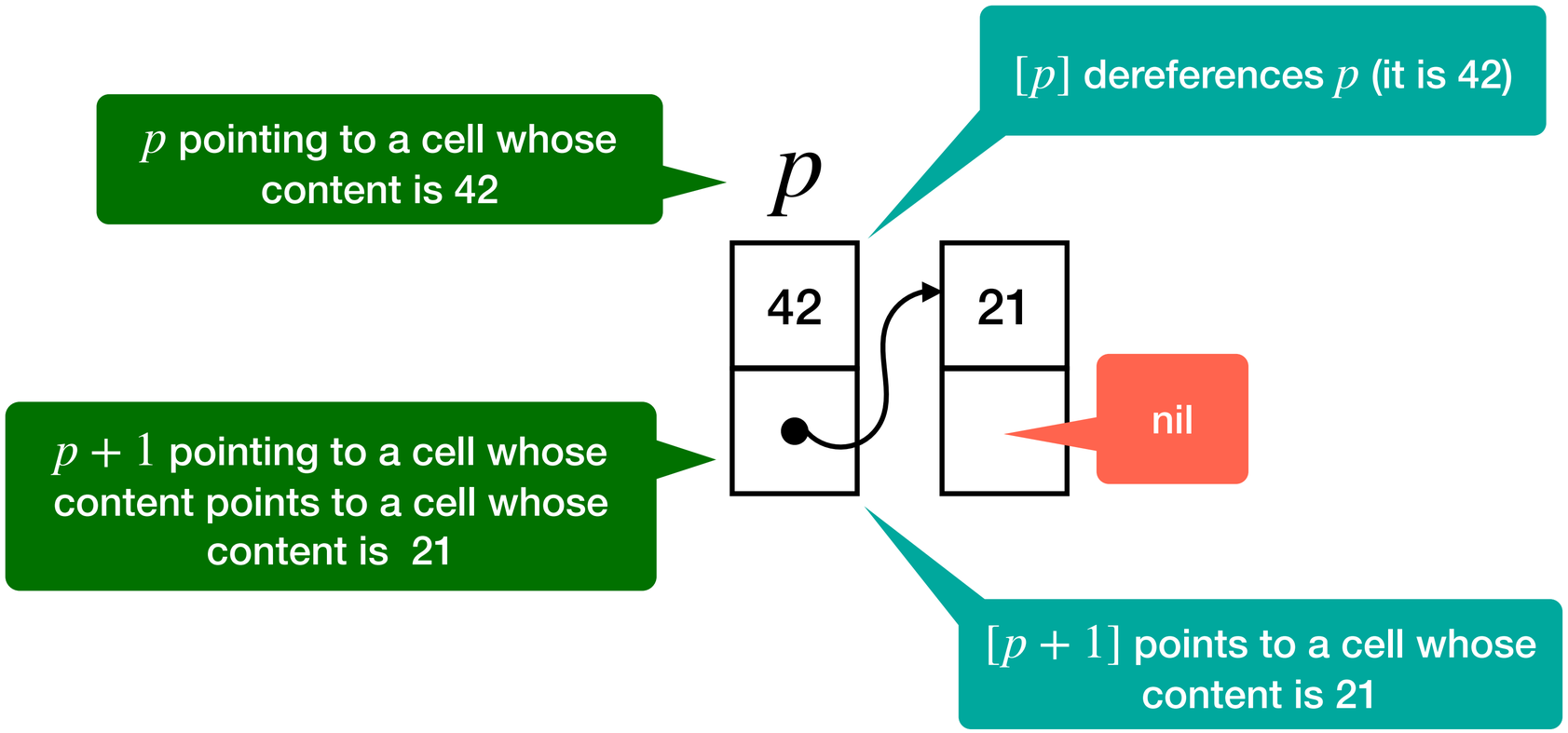
"If the pointer is not pointing to a valid object or function, bad things may happen"

Robert C. Seacord

Effective C:
An Introduction
to Professional C
Programming

ChatGPT imaginery

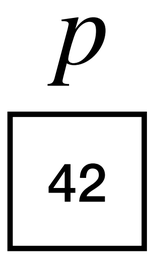
Pointer notation



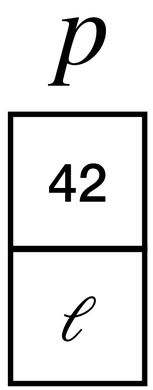
e ::= skip
| $b?$
| $x := a$
| $x := [a]$ // read
| $[a_1] := a_2$ // write
| $x :=$ alloc( )
| free(x)

The predicate "points to"

$p \mapsto 42$

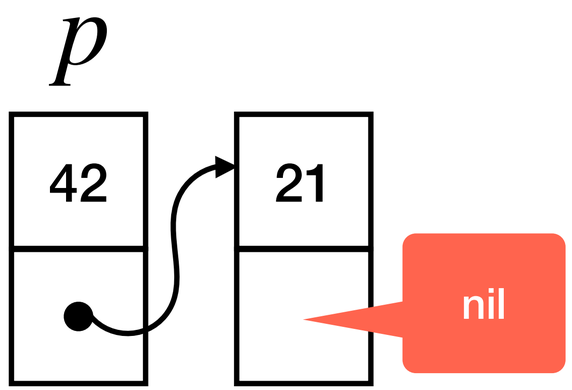


$p \mapsto \langle 42, \ell \rangle$



$p \mapsto \langle 42, \ell \rangle$

$\ell \mapsto \langle 21, \text{nil} \rangle$



Starter example

$\{\text{true}\}$

$[x] := 1;$

$[y] := 2;$

$[z] := 3;$

$\{z \mapsto 3\}$

is it valid?

Starter example

$\{\text{true}\}$

$[x] := 1;$

$[y] := 2;$

$[z] := 3;$

$\{z \mapsto 3\}$

valid!

Starter example

$\{\mathsf{true}\}$

$[x] := 1;$

$[y] := 2;$

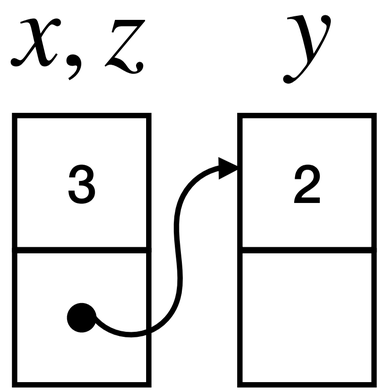
$[z] := 3;$

$\{x \mapsto 1 \wedge y \mapsto 2 \wedge z \mapsto 3\}$

is it valid?

Starter example

$$\{\text{true}\}$$
$$[x] := 1;$$
$$[y] := 2;$$
$$[z] := 3;$$
$$\{x \mapsto 1 \land y \mapsto 2 \land z \mapsto 3\}$$



we must exclude aliasing!

Starter example

$\{x \neq y \wedge x \neq z \wedge y \neq z\}$

$[x] := 1;$

$[y] := 2;$

$[z] := 3;$

$\{x \mapsto 1 \wedge y \mapsto 2 \wedge z \mapsto 3\}$

valid!

Starter example

$\{x_1 \neq x_2 \wedge \ldots\}$

$[x_1] := 1;$

$[x_2] := 2;$

...

$[x_n] := n;$

$\{x_1 \mapsto 1 \wedge \ldots \wedge x_n \mapsto n\}$

$n(n-1)/2$ conjuncts!

Starter example

$\langle x \neq y \wedge x \neq z \wedge y \neq z \rangle$

$[x] := 1;$

$[y] := 2;$

$[z] := 3;$

$\langle x \mapsto 1 \wedge y \mapsto 2 \wedge z \mapsto 3 \rangle$

is it valid?

Starter example

$\langle x \neq y \wedge x \neq z \wedge y \neq z \rangle$

$[x] := 1;$

$[y] := 2;$

$[z] := 3;$

$\langle x \mapsto 1 \wedge y \mapsto 2 \wedge z \mapsto 3 \rangle$

what if x or y or z are not allocated?

Starter example

ownership of heap cell at x

separating conjunction!

$$\{x_1 \mapsto \_ * \ldots * x_n \mapsto \_\}$$

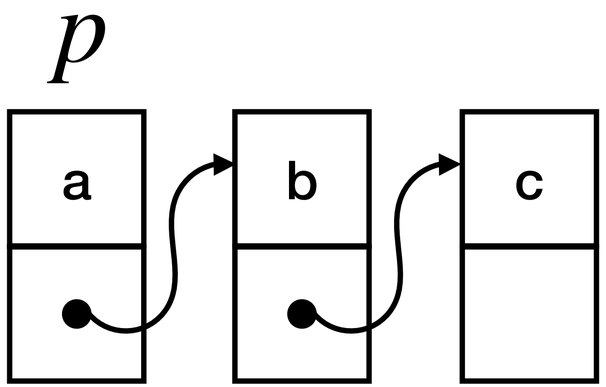
n conjuncts!

$$[x_1] := 1;$$
$$[x_2] := 2;$$
$$\ldots$$
$$[x_n] := n;$$

$$\{x_1 \mapsto 1 * \ldots * x_n \mapsto n\}$$

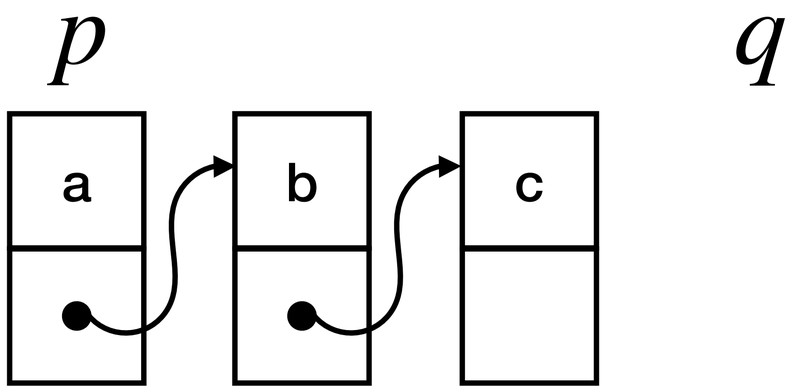
List example

q := nil;
while $p \neq$ nil do (
    $t := [p + 1]$;
    $[p + 1] := q$;
    $q := p$;
    $p := t$
)



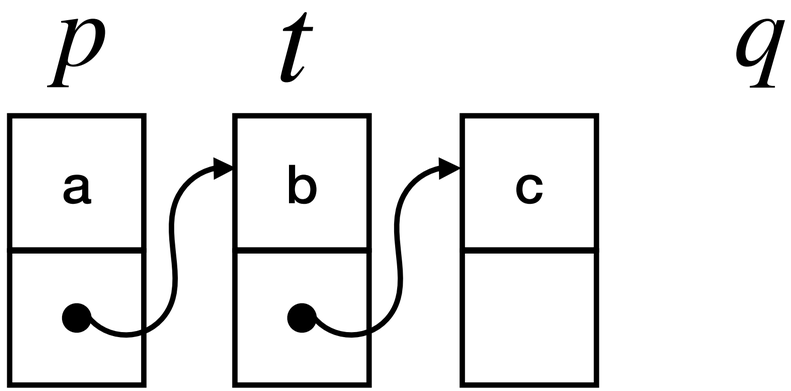
List example

q := nil;
while $p \neq$ nil do (
    $t := [p + 1]$;
    $[p + 1] := q$;
    $q := p$;
    $p := t$
)



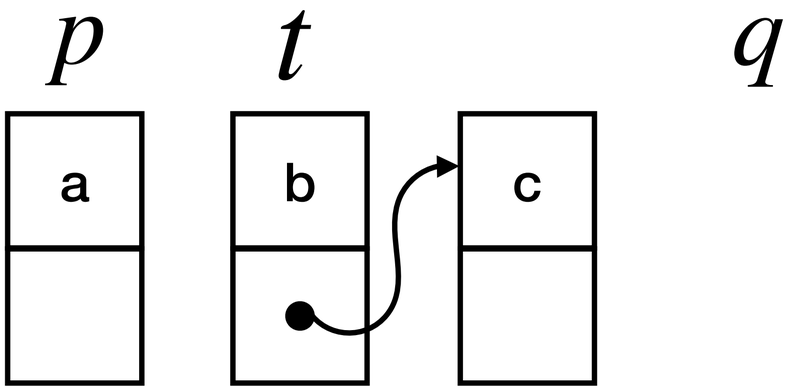
List example

q := nil;
while $p \neq$ nil do (
    $t := [p + 1]$;
    $[p + 1] := q$;
    $q := p$;
    $p := t$
)



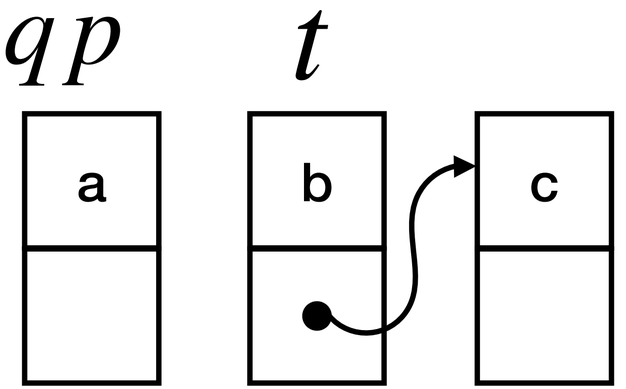
List example

q := nil;
while $p \neq$ nil do (
    $t := [p + 1]$;
    $[p + 1] := q$;
    $q := p$;
    $p := t$
)



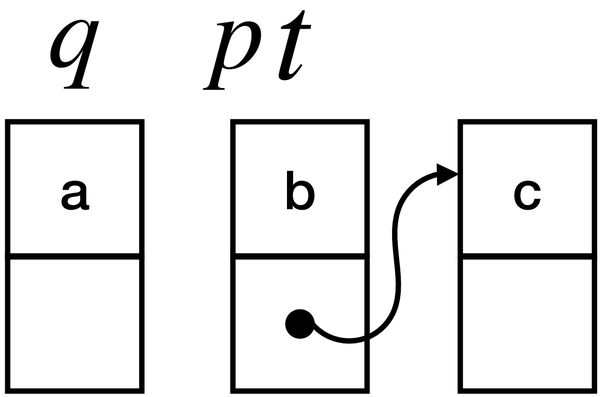
List example

```
q := nil;
while p ≠ nil do (
    t := [p + 1];
    [p + 1] := q;
    q := p;
    p := t
)
```



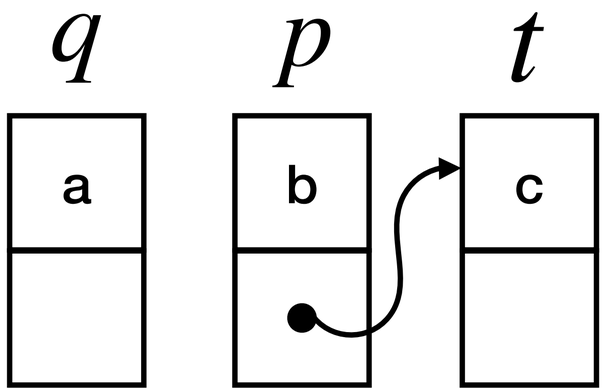
List example

q := nil;
while $p \neq$ nil do (
    $t := [p + 1]$;
    $[p + 1] := q$;
    $q := p$;
    $p := t$
)



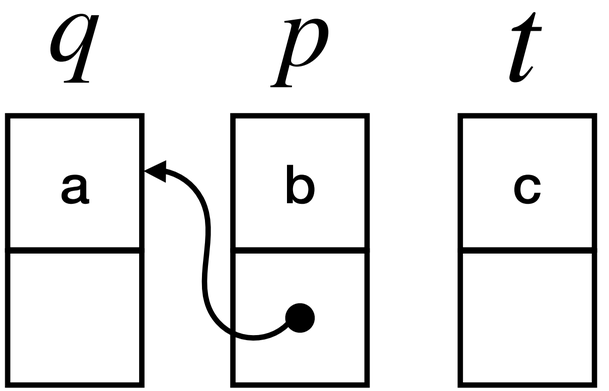
List example

q := nil;
while $p \neq$ nil do (
    $t := [p + 1]$;
    $[p + 1] := q$;
    $q := p$;
    $p := t$
)



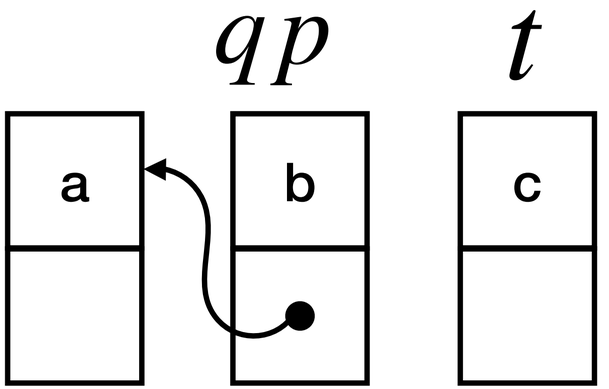
List example

q := nil;
while $p \neq$ nil do (
    $t := [p + 1]$;
    $[p + 1] := q$;
    $q := p$;
    $p := t$
)



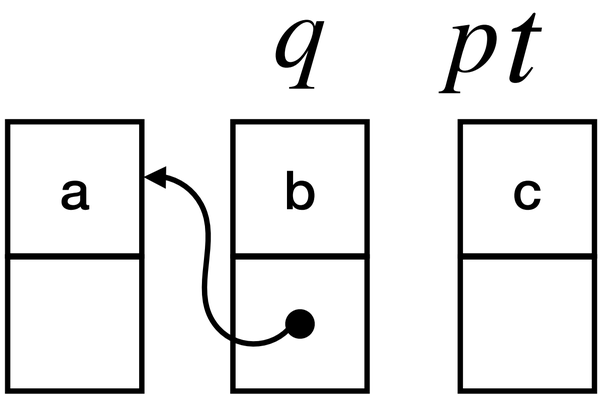
List example

q := nil;
while $p \neq$ nil do (
    $t := [p + 1]$;
    $[p + 1] := q$;
    $q := p$;
    $p := t$
)



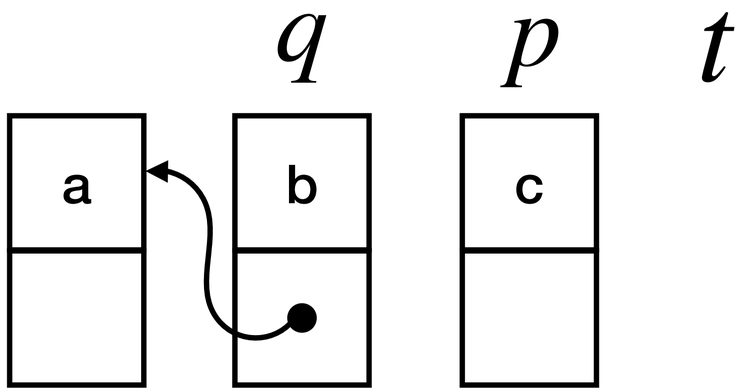
List example

q := nil;
while $p \neq$ nil do (
    $t := [p + 1]$;
    $[p + 1] := q$;
    $q := p$;
    $p := t$
)



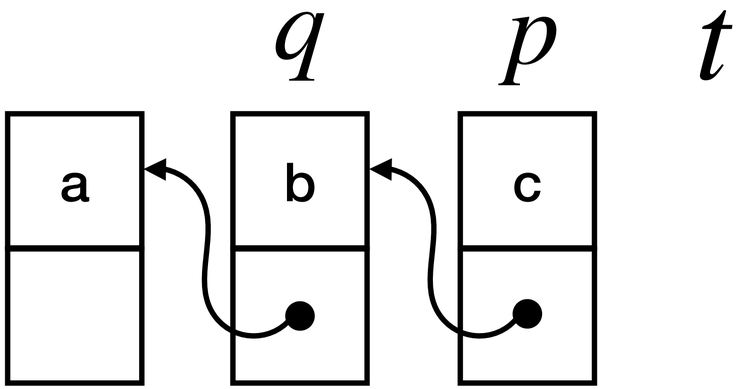
List example

q := nil;
while $p \neq$ nil do (
    $t := [p + 1]$;
    $[p + 1] := q$;
    $q := p$;
    $p := t$
)



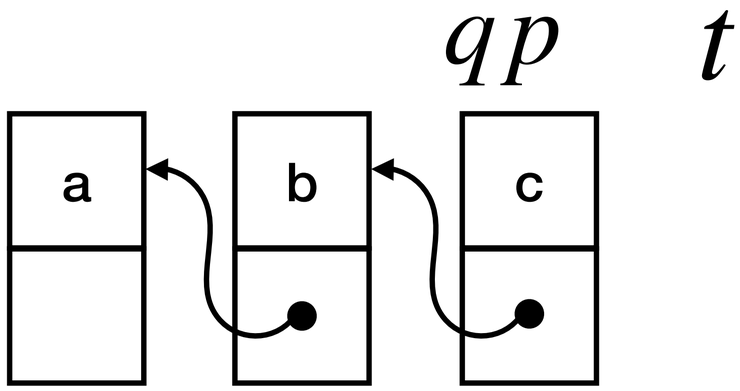
List example

q := nil;
while $p \neq$ nil do (
    $t := [p + 1]$;
    $[p + 1] := q$;
    $q := p$;
    $p := t$
)



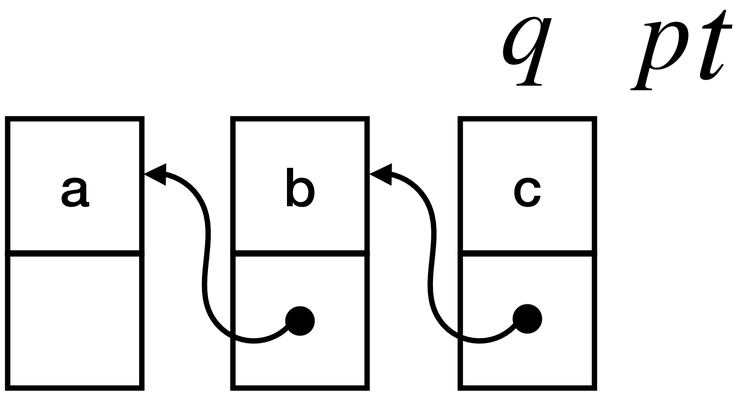
List example

q := nil;
while $p \neq$ nil do (
    $t := [p + 1]$;
    $[p + 1] := q$;
    $q := p$;
    $p := t$
)



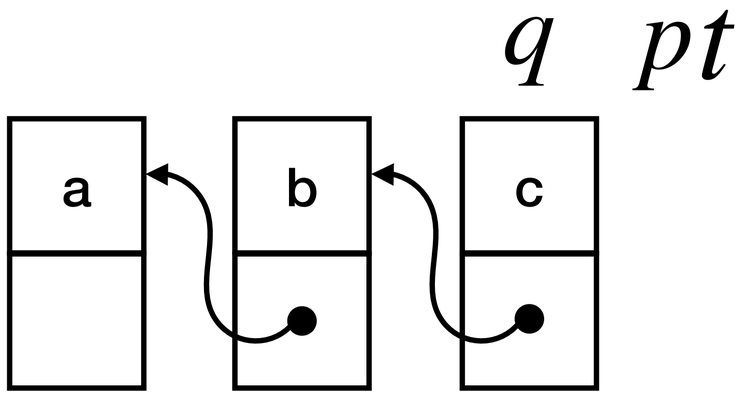
List example

q := nil;
while $p \neq$ nil do (
    $t := [p + 1]$;
    $[p + 1] := q$;
    $q := p$;
    $p := t$
)



List example

q := nil;
while $p \neq$ nil do (
    $t := [p + 1]$;
    $[p + 1] := q$;
    $q := p$;
    $p := t$
)



List reversal example

$q := \text{nil};$
while $p \neq \text{nil}$ do (
    $t := [p + 1];$
    $[p + 1] := q;$
    $q := p;$
    $p := t$
)

invariant?

intuitively

q points to the already reversed list fragment β

p points to the list fragment α still to be reversed

let $\alpha_0^\dagger$ be the reverse of the original list α_0

it is given by juxtaposing $\alpha^\dagger$ (the reverse of α) with β

List reversal example

$q := \text{nil};$
while $p \neq \text{nil}$ do (
  $t := [p + 1];$
  $[p + 1] := q;$
  $q := p;$
  $p := t$
)

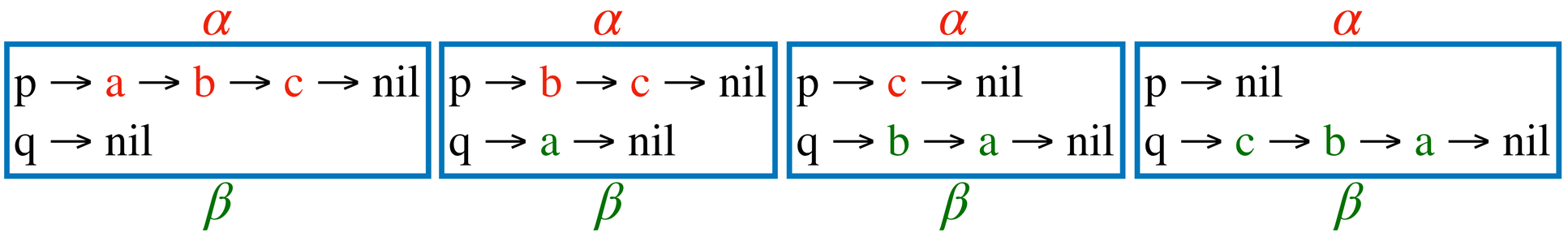
invariant?

reversal

$$P \triangleq \exists \alpha, \beta \,.\, \text{list}(\alpha, p) \wedge \text{list}(\beta, q) \wedge \alpha_0^\dagger = \alpha^\dagger \cdot \beta$$

part to be reversed

already reversed



an inductive list predicate:

points to

$$\text{list}(\epsilon, p) \triangleq (p = \text{nil})$$

$$\text{list}(v \cdot \alpha, p) \triangleq \exists q \,.\, p \mapsto \langle v, q \rangle \wedge \text{list}(\alpha, q)$$

List reversal example

$q := \text{nil};$
while $p \neq \text{nil}$ do (
  $t := [p + 1];$
  $[p + 1] := q;$
  $q := p;$
  $p := t$
)

invariant?

reversal

$P \triangleq \exists \alpha, \beta\,.\ \text{list}(\alpha, p) \wedge \text{list}(\beta, q) \wedge \alpha_0^\dagger = \alpha^\dagger \cdot \beta$

would fail if lists overlap!

List reversal example

$q := \text{nil};$
while $p \neq \text{nil}$ do (
  $t := [p + 1];$
  $[p + 1] := q;$
  $q := p;$
  $p := t$
)

invariant?

$P \triangleq \exists \alpha, \beta \,.\, \text{list}(\alpha, p) \wedge \text{list}(\beta, q) \wedge \alpha_0^\dagger = \alpha^\dagger \cdot \beta$
$\wedge\, (\forall k \,.\, \text{reach}(p, k) \wedge \text{reach}(q, k) \Rightarrow k = \text{nil})$

do not overlap

an inductive reachability predicate:

$\text{reach}(p, q) \triangleq p = q$
$\vee\ \exists v, t \,.\, p \mapsto \langle v, t \rangle \wedge \text{reach}(t, q)$

List reversal example

$q := \text{nil};$
while $p \neq \text{nil}$ do (
  $t := [p + 1];$
  $[p + 1] := q;$
  $q := p;$
  $p := t$
)

invariant?

$P \triangleq \exists \alpha, \beta\,.\ \text{list}(\alpha, p) \wedge \text{list}(\beta, q) \wedge \alpha_0^\dagger = \alpha^\dagger \cdot \beta$

$\wedge\ (\forall k\,.\ \text{reach}(p, k) \wedge \text{reach}(q, k) \Rightarrow k = \text{nil})$

what if other lists share some nodes with p?

List reversal example

$q := \text{nil};$
$\text{while } p \neq \text{nil do (}$
$\quad t := [p+1];$
$\quad [p+1] := q;$
$\quad q := p;$
$\quad p := t$
)

invariant?

$$P \triangleq \exists \alpha, \beta\,.\ \text{list}(\alpha, p) \wedge \text{list}(\beta, q) \wedge \text{list}(\gamma, x) \wedge \alpha_0^\dagger = \alpha^\dagger \cdot \beta$$
$$\wedge\ (\forall k\,.\ \text{reach}(p,k) \wedge \text{reach}(q,k) \Rightarrow k = \text{nil})$$
$$\wedge\ (\forall k\,.\ \text{reach}(x,k) \wedge (\text{reach}(p,k) \vee \text{reach}(q,k)) \Rightarrow k = \text{nil})$$

List reversal example

$q := \text{nil};$
while $p \neq \text{nil}$ do (
  $t := [p + 1];$
  $[p + 1] := q;$
  $q := p;$
  $p := t$
)

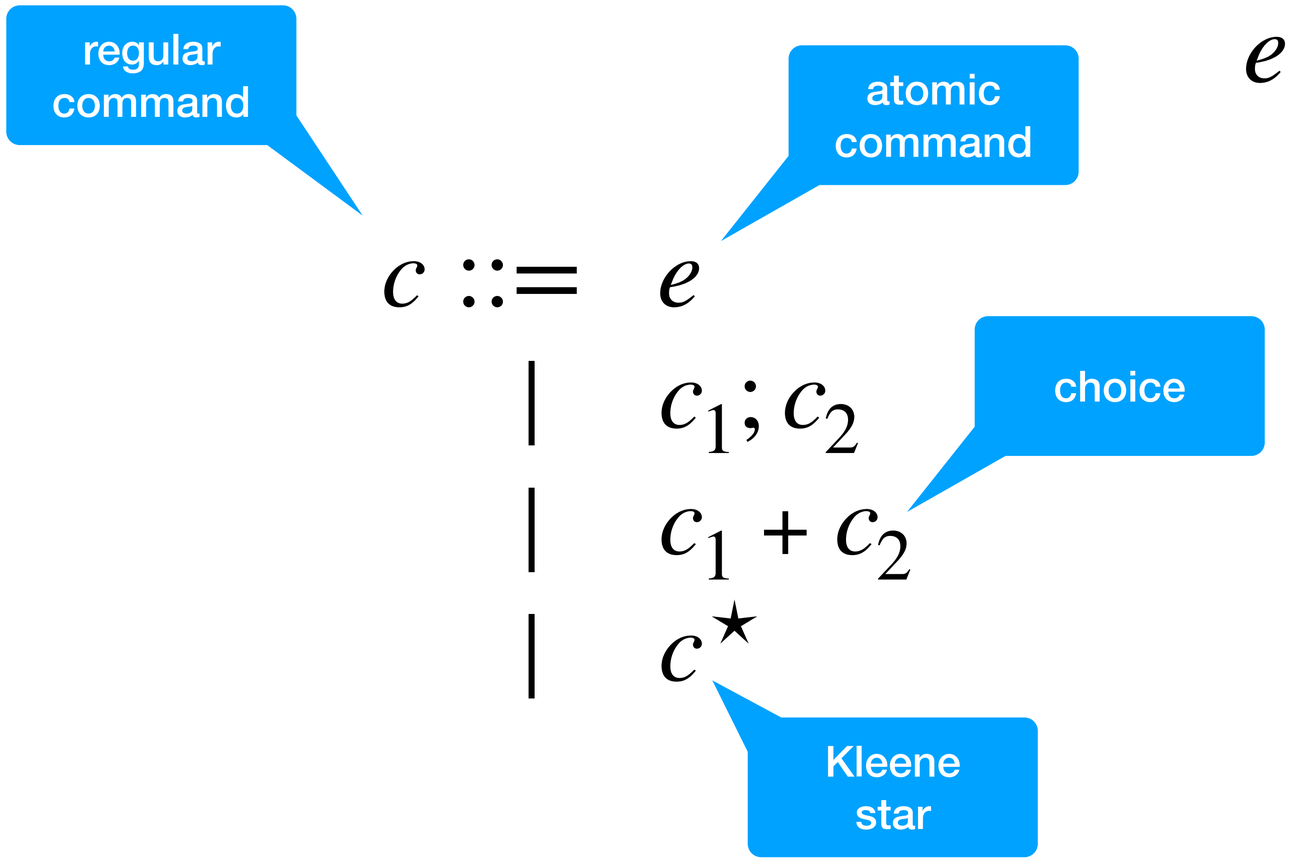
$P \triangleq \exists \alpha, \beta \,.\, (\text{list}(\alpha, p) * \text{list}(\beta, q)) \wedge \alpha_0^\dagger = \alpha^\dagger \cdot \beta$

separating conjunction!

the two lists speak about different parts of the heap

Heap manipulating atomic commands

Regular commands



$$
\begin{aligned}
e ::= \ & \mathsf{skip} \\
\mid \ & b? \\
\mid \ & x := a \\
\mid \ & x := [a] && \text{// read} \\
\mid \ & [a_1] := a_2 && \text{// write} \\
\mid \ & x := \mathsf{alloc}() \\
\mid \ & \mathsf{free}(x) \\
\mid \ & x := \mathsf{cons}(a_0, \ldots, a_k)
\end{aligned}
$$

Stores, heaps and states

store

set of variables

set of values

$$s : X \to \mathbb{Z}$$

heap

set of locations

set of values

$$h : \mathbb{N} \rightharpoonup_{\text{fin}} \mathbb{Z}_\perp$$

null=-1

special value (deallocated)

set of all stores

$$\text{Stores} \triangleq \{s : X \to \mathbb{Z}\}$$

set of all heaps

$$\text{Heaps} \triangleq \{h : \mathbb{N} \rightharpoonup_{\text{fin}} \mathbb{Z}_\perp\}$$

state

store-heap pair

$$\sigma = \langle s, h \rangle$$

set of all states

$$\text{States} \triangleq \text{Stores} \times \text{Heaps}$$

concrete domain

$$\wp(\text{States})$$

Notation

$\mathrm{dom}(f)$ is the domain of definition of a heap function

$h_1 \# h_2 \triangleq \mathrm{dom}(h_1) \cap \mathrm{dom}(h_2) = \varnothing$

disjoint heaps

$h_1 \bullet h_2$ is the union of functions with disjoint domains

(disjoint) heap composition

(undefined if $\neg(h_1 \# h_2)$)

$f[x \mapsto n]$ is the partial function like f except that x goes to n

update

Assertion language

Assertion language

assertion

$$
\begin{aligned}
P ::= \quad & \text{true} \mid \text{false} \mid a_1 < a_2 \mid a_1 = a_2 \mid \ldots \\
\mid \quad & \neg P \mid P_1 \wedge P_2 \mid \exists x\,.\,P \mid \ldots \\
\mid \quad & \text{emp} \\
\mid \quad & a_1 \mapsto a_2 \\
\mid \quad & P_1 * P_2
\end{aligned}
$$

Boolean and classical assertions

also called pure assertions

empty heap

structural assertions

ownership

and separately

Satisfaction: classical

$$\langle s, h \rangle \vDash P$$

the assertion P holds for the given state $\langle s, h \rangle$

$\langle s, h \rangle \vDash a_1 < a_2$ iff $s \vDash a_1 < a_2$

$\langle s, h \rangle \vDash P_1 \wedge P_2$ iff $\langle s, h \rangle \vDash P_1$ and $\langle s, h \rangle \vDash P_2$

$\langle s, h \rangle \vDash \forall x \,.\, P$ iff $\forall v \in \mathbb{Z} \,.\, \langle s[x \mapsto v], h \rangle \vDash P$

...

Satisfaction: structural

$\langle s, h \rangle \models P$

the assertion P holds
for the given state $\langle s, h \rangle$

$\langle s, h \rangle \models a_1 \mapsto a_2$ iff $\mathrm{dom}(h) = \{[\![a_1]\!]s\}$ and $h([\![a_1]\!]s) = [\![a_2]\!]s$

$\langle s, h \rangle \models \mathrm{emp}$ iff h is the empty map [ ]

$\langle s, h \rangle \models P_1 * P_2$ iff $\exists h_1, h_2 \,.\, \langle s, h_1 \rangle \models P_1$ and $\langle s, h_2 \rangle \models P_2$ and $h = h_1 \bullet h_2$

splits the heap
but not the store!

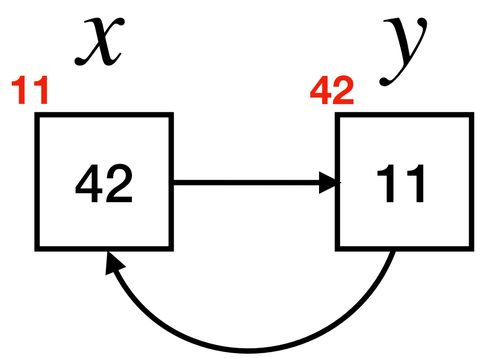
Example

two locations

$\text{dom}(h) = \{11,42\}$

$s(x) = 11 \quad h(11) = 42$

$s(y) = 42 \quad h(42) = 11$



$h = h_1 \bullet h_2$

$\text{dom}(h_1) = \{11\}$

$h_1(11) = 42$

$\text{dom}(h_2) = \{42\}$

$h_2(42) = 11$

$\langle s, h \rangle \vDash \text{emp}$? ✗

$\langle s, h \rangle \vDash x \mapsto y$? ✗

$\langle s, h \rangle \vDash x \mapsto y * y \mapsto x$? ✓

$\langle s, h_1 \rangle \vDash x \mapsto y$

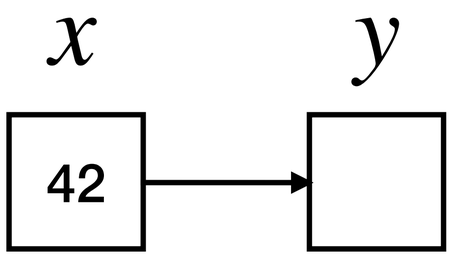
$\langle s, h_2 \rangle \vDash y \mapsto x$

Example

$\mathrm{dom}(h_1) = \{11\}$

$s(x) = 11 \qquad h_1(11) = 42$

$s(y) = 42$



$\langle s, h_1 \rangle \vDash \mathrm{emp}$?

$\langle s, h_1 \rangle \vDash x \mapsto y$?

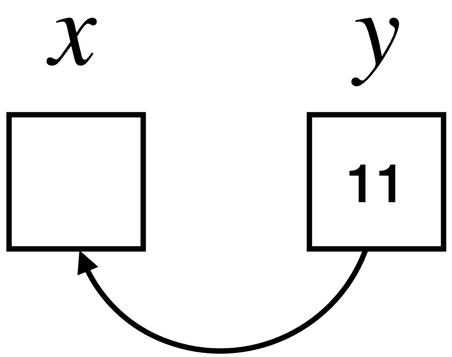
$\langle s, h_1 \rangle \vDash x \mapsto y * y \mapsto x$?

Example

$\mathrm{dom}(h_2) = \{42\}$

$s(x) = 11$

$s(y) = 42 \qquad h_2(42) = 11$



$\langle s, h_2 \rangle \vDash \mathsf{emp}$?

$\langle s, h_2 \rangle \vDash y \mapsto x$?

$\langle s, h_2 \rangle \vDash x \mapsto y * y \mapsto x$?

Example

$\text{dom}(h_1) = \{11\}$

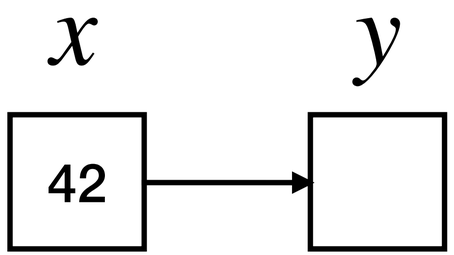
$s(x) = 11 \quad h_1(11) = 42$

$s(y) = 42$

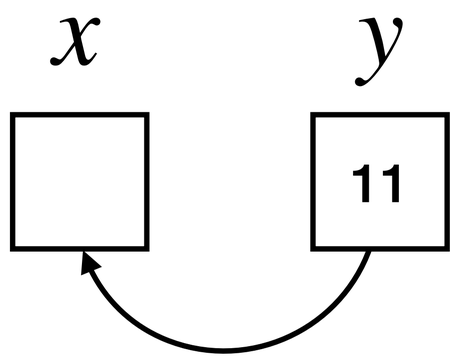
$\text{dom}(h_2) = \{42\}$

$s(x) = 11$

$s(y) = 42 \quad h_2(42) = 11$



$h = h_1 \bullet h_2$



$\langle s, h \rangle \vDash x \mapsto y * y \mapsto x$?

Some subtleties

any assertion

$P * \text{emp} \equiv P$

pure assertion

$P \wedge \text{emp} \not\equiv P$

any heap

single-cell heap

$x \mapsto v * \text{true} \not\equiv x \mapsto v$

$x \mapsto v * P \not\equiv x \mapsto v \wedge P$

cannot separate

$x \mapsto v * x \mapsto w \equiv \text{false}$

$x \mapsto v \wedge x \mapsto w \equiv x \mapsto v \wedge (v = w)$

$(x = y) * (x = y) \equiv (x = y)$

$P * P \equiv P$

$x \mapsto v * y \mapsto w \not\Rightarrow x \mapsto v$

$x \mapsto v \wedge y \mapsto w \Rightarrow x \mapsto v$

$x \mapsto v \not\Rightarrow x \mapsto v * y \mapsto w$

$x \mapsto v \not\Rightarrow x \mapsto v \wedge y \mapsto w$

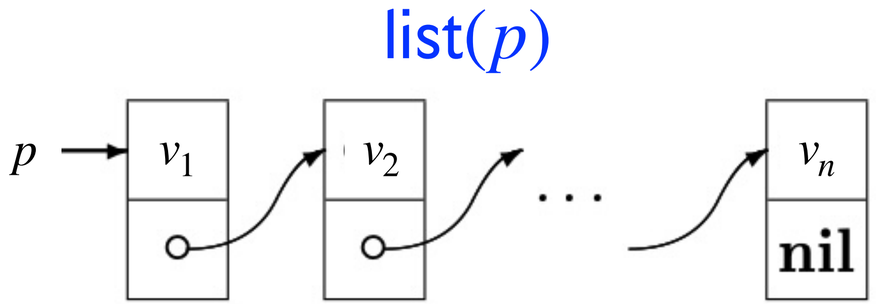
Example: list segments

Let us define the following inductive predicate for **list segments**

$$\mathsf{ls}(a_1, a_2, 0) \triangleq a_1 = a_2 \wedge \mathsf{emp}$$

$$\mathsf{ls}(a_1, a_2, n+1) \triangleq a_1 \neq a_2 \wedge \exists \ell \,.\, a_1 \mapsto \ell * \mathsf{ls}(\ell, a_2, n)$$

and let $\mathsf{ls}(a_1, a_2) \triangleq \exists n \,.\, \mathsf{ls}(a_1, a_2, n)$ and $\mathsf{list}(a) \triangleq \mathsf{ls}(a, \mathsf{nil})$



Example: list segments

Let us define the following inductive predicate for **list segments**

$$\mathsf{ls}(a_1, a_2, 0) \triangleq a_1 = a_2 \wedge \mathsf{emp}$$
$$\mathsf{ls}(a_1, a_2, n+1) \triangleq a_1 \neq a_2 \wedge \exists \ell \,.\, a_1 \mapsto \ell * \mathsf{ls}(\ell, a_2, n)$$

and let $\mathsf{ls}(a_1, a_2) \triangleq \exists n \,.\, \mathsf{ls}(a_1, a_2, n)$ and $\mathsf{list}(a) \triangleq \mathsf{ls}(a, \mathsf{nil})$

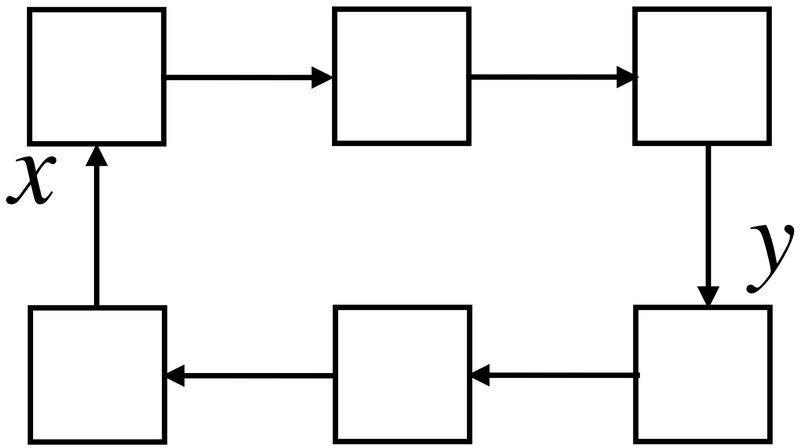
Does the heap in the figure satisfy $\mathsf{ls}(x, x)$? ✗

and $\mathsf{ls}(x, y) * \mathsf{ls}(y, x)$? ✓

and $\mathsf{list}(x)$? ✗

and $\mathsf{ls}(x, y, 3)$? ✗

and $\exists \ell \,.\, \mathsf{ls}(x, \ell)$? ✗



Separation Logic (SL)

CSL 2001

Local Reasoning about Programs that Alter Data Structures

Peter O'Hearn[1], John Reynolds[2], and Hongseok Yang[3]

[1] Queen Mary, University of London
[2] Carnegie Mellon University
[3] University of Birmingham and University of Illinois at Urbana-Champaign

Abstract. We describe an extension of Hoare's logic for reasoning about programs that alter data structures. We consider a low-level storage model based on a heap with associated lookup, update, allocation and deallocation operations, and unrestricted address arithmetic. The assertion language is based on a possible worlds model of the logic of bunched implications, and includes spatial conjunction and implication connectives alongside those of classical logic. Heap operations are axiomatized using what we call the "small axioms", each of which mentions only those cells accessed by a particular command. Through these and a number of examples we show that the formalism supports local reasoning: A specification and proof can concentrate on only those cells in memory that a program accesses.
This paper builds on earlier work by Burstall, Reynolds, Ishtiaq and O'Hearn on reasoning about data structures.

1 Introduction

Pointers have been a persistent trouble area in program proving. The main difficulty is not one of finding an in-principle adequate axiomatization of pointer operations; rather there is a mismatch between simple intuitions about the way that pointer operations work and the complexity of their axiomatic treatments. For example, pointer assignment is operationally simple, but when there is aliasing, arising from several pointers to a given cell, then an alteration to that cell may affect the values of many syntactically unrelated expressions. (See [20, 2, 4, 6] for discussion and references to the literature on reasoning about pointers.)

We suggest that the source of this mismatch is the global view of state taken in most formalisms for reasoning about pointers. In contrast, programmers reason informally in a local way. Data structure algorithms typically work by applying local surgeries that rearrange small parts of a data structure, such as rotating a small part of a tree or inserting a node into a list. Informal reasoning usually concentrates on the effects of these surgeries, without picturing the entire memory of a system. We summarize this local reasoning viewpoint as follows.

> To understand how a program works, it should be possible for reasoning and specification to be confined to the cells that the program actually accesses. The value of any other cell will automatically remain unchanged.

L. Fribourg (Ed.): CSL 2001, LNCS 2142, pp. 1–19, 2001.

"it should be possible for reasoning and specification to be confined to the cells that the program actually accesses. The value of any other cell will automatically remain unchanged"



Separation Logic principles

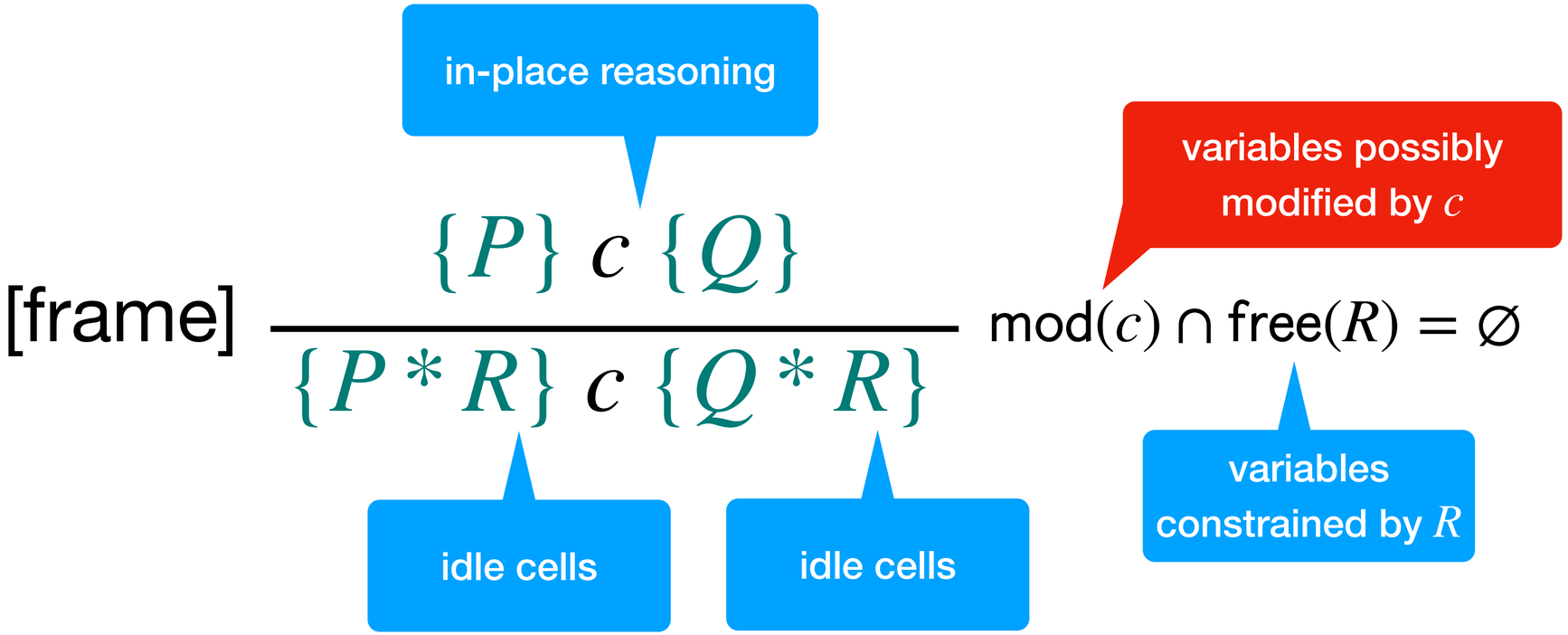
Over-approximation logic for proving correctness

Separation logic = local axioms + frame rule

Local axioms

$$\frac{}{\{P\}\ \textit{atomic command}\ \{Q\}}$$

Frame rule


in-place reasoning
variables possibly modified by c
[frame]
$\{P\}\ c\ \{Q\}$
$\{P * R\}\ c\ \{Q * R\}$
$\mathrm{mod}(c) \cap \mathrm{free}(R) = \varnothing$
idle cells
idle cells
variables constrained by R

Modified variables

$mod(skip) = \emptyset$

$mod(b?) = \emptyset$

$mod(free(x)) = \emptyset$

$mod([x] \coloneqq y) = \emptyset$

$mod(c_1 + c_2) = mod(c_1) \cup mod(c_2)$

$mod(x \coloneqq a) = \{x\}$

$mod(x \coloneqq alloc()) = \{x\}$

$mod(x \coloneqq [y]) = \{x\}$

$mod(c_1; c_2) = mod(c_1) \cup mod(c_2)$

$mod(c^*) = mod(c)$

Note that $\boldsymbol{free(x)}$ and $[\boldsymbol{x}] \coloneqq \boldsymbol{y}$ do not modify x:
this is because they only modify the value pointed by x, not the actual value of x

Free variables

pure assertions

$fv(true) = \emptyset$

$fv(false) = \emptyset$

$fv(a_1 = a_2) = fv(a_1) \cup fv(a_2)$

$fv(p \wedge q) = fv(p) \cup fv(q)$

...

$fv(\exists x.\, p) = fv(p) \backslash \{x\}$

$fv(emp) = \emptyset$

$fv(x \mapsto \mathsf{a}) = \{x\} \cup fv(a)$

$fv(p * q) = fv(p) \cup fv(q)$

structural assertions

Some abbreviations

$$a \mapsto \_ \triangleq \exists v \,.\, a \mapsto v \qquad \text{(for } v \text{ fresh)}$$

$$a_1 \doteq a_2 \triangleq (a_1 = a_2) \wedge \mathsf{emp}$$

$$a \mapsto \langle a_0, \ldots, a_k \rangle \triangleq (a \mapsto a_0) * \ldots * (a + k \mapsto a_k)$$

Local axioms: write

[write] $$\frac{}{\{???\}\ [a_1] := a_2\ \{???\}}$$

Local axioms: write

[write] $\dfrac{}{\{a_1 \mapsto \_\}\ [a_1] := a_2\ \{???\}}$

$a \mapsto \_ \triangleq \exists v.\ a \mapsto v$

Local axioms: write

[write] $$\frac{}{\{a_1 \mapsto \_\}\ [a_1] := a_2\ \{a_1 \mapsto a_2\}}$$

$$\{???\}\ [x] := y\ \{???\}$$

Local axioms: write

[write] $$\frac{}{\{a_1 \mapsto \_\}\ [a_1] := a_2\ \{a_1 \mapsto a_2\}}$$

$$\{x \mapsto \_\}\ [x] := y\ \{x \mapsto y\}$$

Local axioms: read

[read] $$\frac{}{\{a \mapsto v \wedge x = x'\}\ x := [a]\ \{x = v \wedge a[x'/x] \mapsto v\}}$$

not strictly necessary

$$\{y \mapsto v\}\ x := [y]\ \{x = v \wedge y \mapsto v\}$$

Local axioms: allocation

$$\text{[alloc]}\ \frac{}{\{\text{emp}\}\ x := \text{alloc}(\,)\ \{x \mapsto \_\}}$$

Local axioms: dispose

$$\text{[dispose]}\ \frac{}{\{a \mapsto \_\}\ \mathsf{free}(a)\ \{\mathsf{emp}\}}$$

$$\{x \mapsto \_\}\ \mathsf{free}(x)\ \{\mathsf{emp}\}$$

Local axioms: allocation

$$\{x \doteq x'\}\ x := \mathsf{cons}(a_1, \ldots, a_k)\ \{x \mapsto \langle a_1[x'/x], \ldots, a_k[x'/x]\rangle\}$$

$a \mapsto \langle a_0, \ldots, a_k\rangle \triangleq (a \mapsto a_0) * \ldots * (a + k \mapsto a_k)$

Example 1

$\{x \mapsto \_ * y \mapsto \_ * z \mapsto \_\}$

frame rule

write axiom

$\{x \mapsto \_\}$

$[x] := 1;$

$\{x \mapsto 1\}$

$\{x \mapsto 1 * y \mapsto \_ * z \mapsto \_\}$

$[y] := 2;$

$[z] := 3;$

$\{x \mapsto 1 * y \mapsto 2 * z \mapsto 3\}$

$$\dfrac{\dfrac{}{\{x \mapsto \_\}\ [x] := 1\ \{x \mapsto 1\}}\ \text{write}}{\{x \mapsto \_ * R\}\ [x] := 1\ \{x \mapsto 1 * R\}}\ \text{frame}$$

with $R \equiv (y \mapsto \_ * z \mapsto \_)$

Example 1

write axiom

$\{x \mapsto \_\}$

$[x] := 1;$

$\{x \mapsto 1\}$

idle frame

$R_x \equiv y \mapsto \_$

write axiom

$\{y \mapsto \_\}$

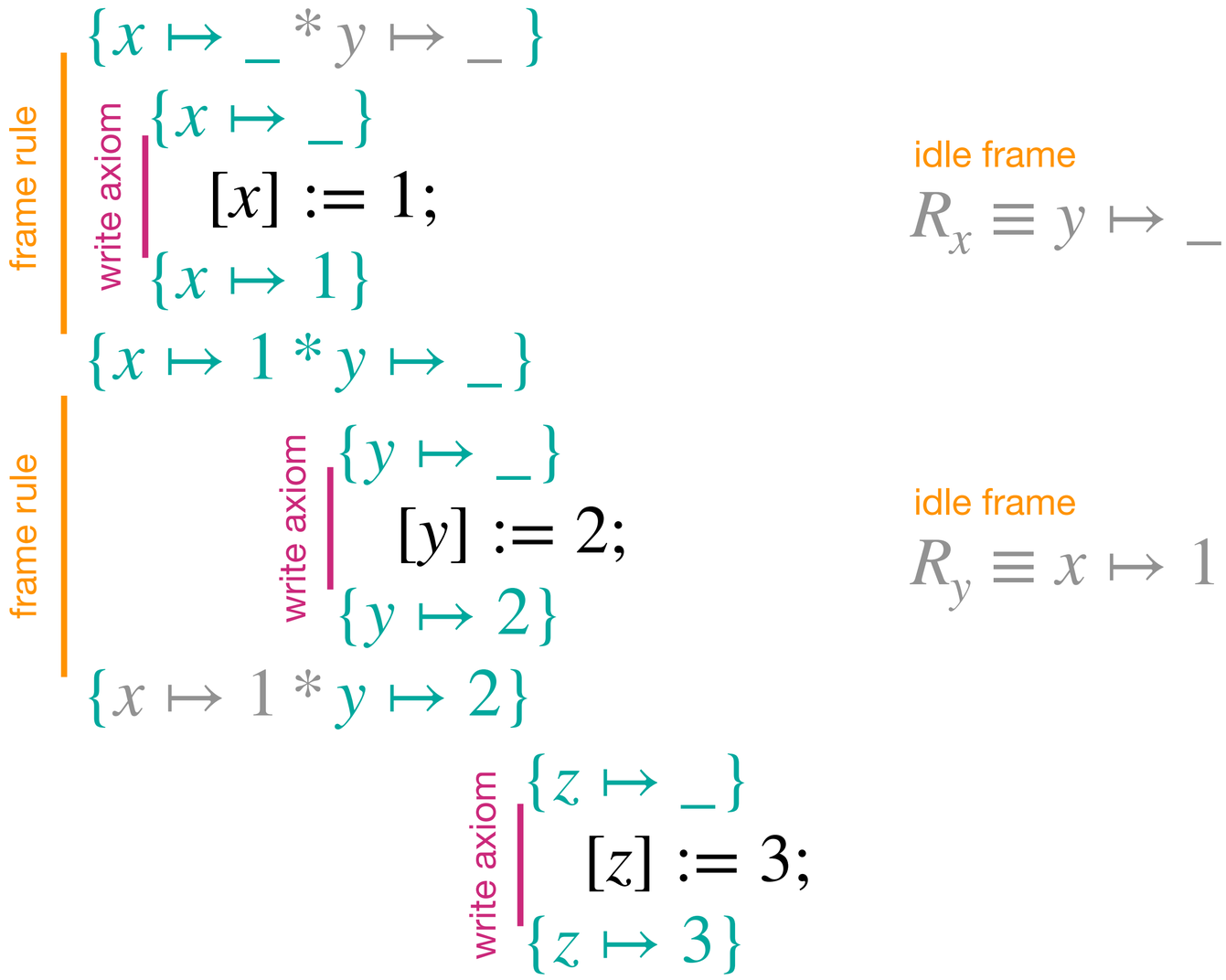
$[y] := 2;$

$\{y \mapsto 2\}$

idle frame

$R_y \equiv x \mapsto 1$

Example 1



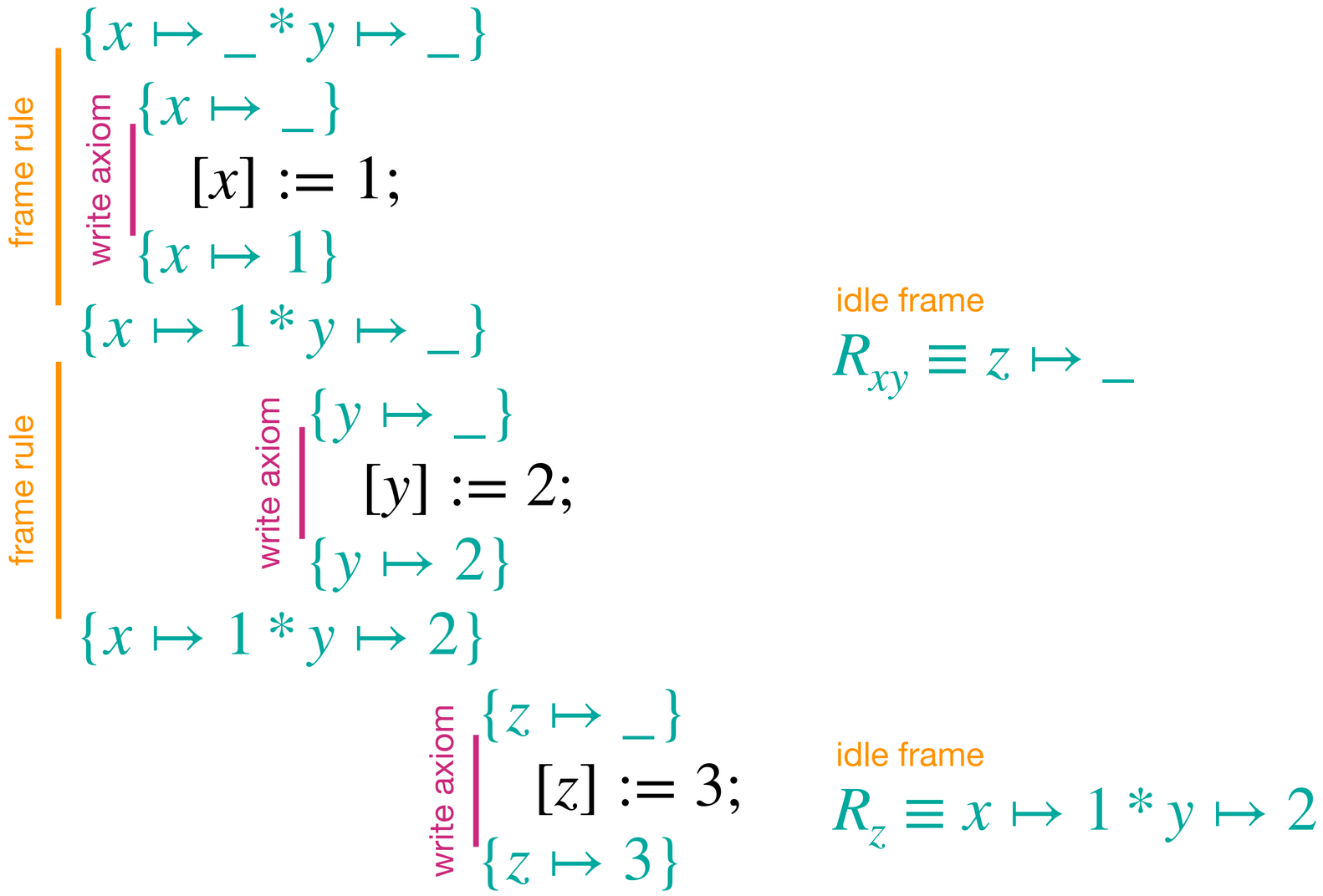
idle frame

$R_x \equiv y \mapsto \_$

idle frame

$R_y \equiv x \mapsto 1$

Example 1



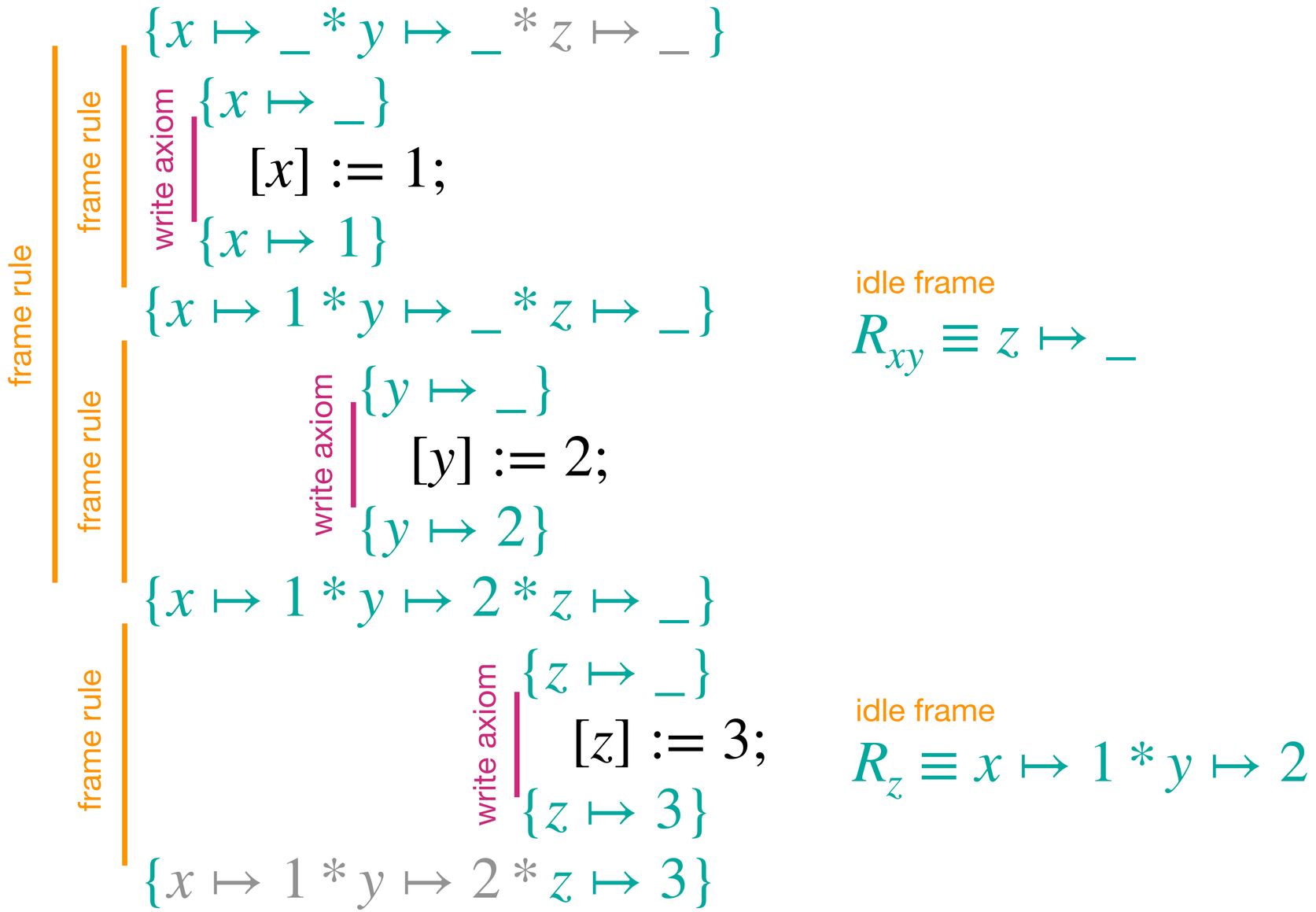
idle frame

$R_{xy} \equiv z \mapsto \_$

idle frame

$R_z \equiv x \mapsto 1 * y \mapsto 2$

Example 1



idle frame

$R_{xy} \equiv z \mapsto \_$

idle frame

$R_z \equiv x \mapsto 1 * y \mapsto 2$

Example 2

Let us consider the following inductive predicates

$$\mathsf{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \mathsf{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell \,.\, a_1 \mapsto \ell * \mathsf{ls}(\ell, a_2))$$

$$\mathsf{list}(a) \triangleq \mathsf{ls}(a, \mathsf{nil})$$

Can we derive this triple?

$$\{\mathsf{list}(x) \wedge x \neq \mathsf{nil}\}\ t := [x]; \mathsf{free}(x); \{\mathsf{list}(t)\}$$

Example 2

$\{x \mapsto v\}\ t := [x];\ \{x \mapsto v \wedge t = v\}$

$\{x \mapsto t\}\ \mathsf{free}(x);\ \{\mathsf{emp}\}$

$\mathsf{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \mathsf{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell\ .\ a_1 \mapsto \ell * \mathsf{ls}(\ell, a_2))$

$\mathsf{list}(a) \triangleq \mathsf{ls}(a, \mathsf{nil})$

Example 2

$\{x \mapsto v * \mathsf{list}(v)\}$

frame

$\quad \{x \mapsto v\}\ t := [x];\ \{x \mapsto v \wedge t = v\}$

$\{(x \mapsto v \wedge t = v) * \mathsf{list}(v)\}$

$\quad \{x \mapsto t\}\ \mathsf{free}(x);\ \{\mathsf{emp}\}$

idle frame

$R_1 \equiv \mathsf{list}(v)$

$\mathsf{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \mathsf{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell\ .\ a_1 \mapsto \ell * \mathsf{ls}(\ell, a_2))$

$\mathsf{list}(a) \triangleq \mathsf{ls}(a, \mathsf{nil})$

Example 2

frame

$\{x \mapsto v * \mathsf{list}(v)\}$

$t := [x];$

$\{(x \mapsto v \wedge t = v) * \mathsf{list}(v)\} \Rightarrow \{x \mapsto t * \mathsf{list}(t)\}$

over-approximation (consequence rule)

$\{x \mapsto t\}\ \mathsf{free}(x);\ \{\mathsf{emp}\}$

$\mathsf{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \mathsf{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell\ .\ a_1 \mapsto \ell * \mathsf{ls}(\ell, a_2))$

$\mathsf{list}(a) \triangleq \mathsf{ls}(a, \mathsf{nil})$

Example 2

$\{x \mapsto v * \mathsf{list}(v)\}$

$t := [x];$

$\{(x \mapsto v \wedge t = v) * \mathsf{list}(v)\} \Rightarrow \{x \mapsto t * \mathsf{list}(t)\}$

over-approximation (consequence rule)

frame | $\{x \mapsto t\}\ \mathsf{free}(x);\ \{\mathsf{emp}\}$

$\{\mathsf{emp} * \mathsf{list}(t)\}$

idle frame

$R_2 \equiv \mathsf{list}(t)$

$\mathsf{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \mathsf{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell\ .\ a_1 \mapsto \ell * \mathsf{ls}(\ell, a_2))$

$\mathsf{list}(a) \triangleq \mathsf{ls}(a, \mathsf{nil})$

Example 2

$\{x \mapsto v * \mathsf{list}(v)\}$

$t := [x];$

$\{x \mapsto t * \mathsf{list}(t)\}$

$\mathsf{free}(x);$

$\{\mathsf{emp} * \mathsf{list}(t)\} \equiv \{\mathsf{list}(t)\}$

Exist rule

$$\frac{\{P\}\ c\ \{Q\}}{\{\exists k\,.\,P\}\ c\ \{\exists k\,.\,Q\}}$$

$\mathsf{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \mathsf{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell\,.\, a_1 \mapsto \ell * \mathsf{ls}(\ell, a_2))$

$\mathsf{list}(a) \triangleq \mathsf{ls}(a, \mathsf{nil})$

Example 2

$\{\exists v\,.\,x \mapsto v * \mathsf{list}(v)\}$

$t := [x];$

$\{x \mapsto t * \mathsf{list}(t)\}$

$\mathsf{free}(x);$

$\{\exists v\,.\,\mathsf{list}(t)\} \equiv \{\mathsf{list}(t)\}$

Exist rule

$$\frac{\{P\}\ c\ \{Q\}}{\{\exists k\,.\,P\}\ c\ \{\exists k\,.\,Q\}}$$

$\mathsf{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \mathsf{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell\,.\,a_1 \mapsto \ell * \mathsf{ls}(\ell, a_2))$

$\mathsf{list}(a) \triangleq \mathsf{ls}(a, \mathsf{nil})$

Example 2

$\{\mathsf{list}(x) \wedge x \neq \mathsf{nil}\} \equiv \{\mathsf{ls}(x, \mathsf{nil}) \wedge x \neq \mathsf{nil}\}$

$\{\exists v \,.\, x \mapsto v * \mathsf{list}(v)\}$

$t := [x];$

$\{x \mapsto t * \mathsf{list}(t)\}$

$\mathsf{free}(x);$

$\{\exists v \,.\, \mathsf{list}(t)\} \equiv \{\mathsf{list}(t)\}$

$\{\mathsf{list}(t)\}$

$\mathsf{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \mathsf{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell \,.\, a_1 \mapsto \ell * \mathsf{ls}(\ell, a_2))$

$\mathsf{list}(a) \triangleq \mathsf{ls}(a, \mathsf{nil})$

Example 2

$\{\mathsf{ls}(x, \mathsf{nil}) \wedge x \neq \mathsf{nil}\} \equiv \{x \neq \mathsf{nil} \wedge \exists v \,.\, x \mapsto v * \mathsf{ls}(v, \mathsf{nil})\}$

$\equiv \{\exists v \,.\, x \mapsto v * \mathsf{list}(v)\}$

$t := [x];$

$\{x \mapsto t * \mathsf{list}(t)\}$

$\mathsf{free}(x);$

$\{\exists v \,.\, \mathsf{list}(t)\} \equiv \{\mathsf{list}(t)\}$

$\{\mathsf{list}(t)\}$

$\mathsf{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \mathsf{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell \,.\, a_1 \mapsto \ell * \mathsf{ls}(\ell, a_2))$

$\mathsf{list}(a) \triangleq \mathsf{ls}(a, \mathsf{nil})$

Example 2

$\{\mathsf{list}(x) \wedge x \neq \mathsf{nil}\} \equiv \{\exists v\,.\, x \mapsto v * \mathsf{list}(v)\}$

exists

$\{x \mapsto v * \mathsf{list}(v)\}$

frame: $\{x \mapsto v\}\ t := [x];\ \{x \mapsto v \wedge t = v\}$

$\{x \mapsto t * \mathsf{list}(t)\}$

frame: $\{x \mapsto t\}\ \mathsf{free}(x);\ \{\mathsf{emp}\}$

$\{\mathsf{emp} * \mathsf{list}(t)\} \equiv \{\mathsf{list}(t)\}$

$\{\mathsf{list}(t)\}$

$$\dfrac{\dfrac{}{\{x \mapsto v\}\ t := [x]\ \{x \mapsto v \wedge t = v\}}\ (\text{read})}{\{x \mapsto v * \mathsf{list}(v)\}\ t := [x]\ \{x \mapsto t * \mathsf{list}(t)\}}\ (\text{Frame}) \qquad \dfrac{\dfrac{}{\{x \mapsto t\}\ \mathsf{free}(x)\ \{emp\}}\ (\text{dealloc})}{\{x \mapsto t * \mathsf{list}(t)\}\ \mathsf{free}(x)\ \{emp * \mathsf{list}(t)\}}\ (\text{Frame})$$

$$\dfrac{\{x \mapsto v * \mathsf{list}(v)\}\ t := [x]\ \{x \mapsto t * \mathsf{list}(t)\} \qquad \{x \mapsto t * \mathsf{list}(t)\}\ \mathsf{free}(x)\ \{emp * \mathsf{list}(t)\}}{\{x \mapsto v * \mathsf{list}(v)\}\ t := [x];\ \mathsf{free}(x)\ \{\mathsf{list}(t)\}}\ (\text{Seq})$$

$\mathsf{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \mathsf{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell\,.\, a_1 \mapsto \ell * \mathsf{ls}(\ell, a_2))$

$\mathsf{list}(a) \triangleq \mathsf{ls}(a, \mathsf{nil})$

Example: concatenation

$\{x \mapsto v * \mathsf{list}(v) * \mathsf{list}(y)\}$

$t := x;$

$n := [t];$

while $n \neq \mathsf{nil}$ do (

$t := n;$

$n := [t];$

)

$[t] := y;$

$\{\mathsf{list}(x)\}$

Precondition

x and y are pointers to the heads of two (singly linked) lists.

The list pointed to by x is non-empty and each list is finite and null-terminated.

The two lists are disjoint (no node belongs to both lists).

The code appends the list pointed to by y to the end of the non-empty list pointed to by x

Postcondition

A unique (singly linked) list pointed to by x.

$\mathsf{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \mathsf{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell \,.\, a_1 \mapsto \ell * \mathsf{ls}(\ell, a_2))$

$\mathsf{list}(a) \triangleq \mathsf{ls}(a, \mathsf{nil})$

Example: concatenation

$\{x \mapsto v * \mathsf{list}(v) * \mathsf{list}(y)\}$

$t := x;$

$n := [t];$

while $n \neq \mathsf{nil}$ do (

$t := n;$

$n := [t];$

)

$[t] := y;$

$\{\mathsf{list}(x)\}$

Try to work on the derivation for the next time...

$\mathsf{ls}(a_1, a_2) \triangleq (a_1 = a_2 \wedge \mathsf{emp}) \vee (a_1 \neq a_2 \wedge \exists \ell \,.\, a_1 \mapsto \ell * \mathsf{ls}(\ell, a_2))$
$\mathsf{list}(a) \triangleq \mathsf{ls}(a, \mathsf{nil})$